MANUAL ACCEPTANCE + TARGETED REGRESSION

Shared-File Reformalization A Reproducible Test Lab

1 Scope and prerequisites

This guide creates situations the recent changes are meant to handle: retrying one declaration among protected neighbors, completing a needed helper, retaining unfinished work, accepting a changed source statement honestly, and keeping compiler evidence fresh without restarting a healthy Lean server.

The mathematics is elementary on purpose. Failures should expose a workflow, preservation, or evidence problem, rather than a difficult theorem.

Route	Instructions
Essential UI trial	Sections 2–4: shared-file setup and retry B.
Deeper UI trials	Sections 5–7: helper, interruption, source update.
Deterministic checks	Sections 8–9: freshness, queue, review, recovery.
Before/after comparison	Section 10: identical inputs and a result sheet.

1. Use the current local macOS stack with a working reformalization executor. The feature requires a successful `sandbox-exec` capability probe. Docker/Linux or a nested sandbox can make these cases unavailable. Record that as *blocked by environment*, never as a passing test.

2. Start the existing stack with `./start-dev.sh -keep-data`. Load the updated unpacked Overleaf extension and use its Lean pane. The same backend project is inspectable in standalone at `http://localhost:5173`.

3. Choose one model and fixed turn/time/spend allowances; record them. Only UI formalization trials call a model. The regression tests below use local fixtures or scripted solver events.

4. Create an Overleaf project named **RetryLab**. Keep baseline copies outside its managed proof directory. Use a fresh baseline for each recipe.

1.1 How to judge the result

A file can compile while containing `sorry`. Completion requires evidence for the selected declaration and its transitive trust closure. Code preservation does not create a personal audit or transfer an old approval. Judge the selected target, protected source, and current evidence separately. If a timing-dependent trial never reaches its required interleaving, record it as *inconclusive*.

2 Create the source targets

One setup pass. Do not let the model choose the dependency structure.

1. In the disposable Overleaf project, replace `main.tex` with the provided `fixtures/overleaf-source.tex`. The complete source is:

```
\documentclass{article}
\usepackage{amsthm}
\newtheorem{theorem}{Theorem}
\begin{document}

\begin{theorem}\label{thm:retry-b}
% lea: formalize label=retry_b
For every natural number $n$, $n+0=n$.
\end{theorem}

\begin{theorem}\label{thm:retry-consumer}
% lea: formalize label=retry_consumer uses={retry_b}
For every natural number $n$, $n+0=n$.
\end{theorem}

\end{document}
```

2. Open the Lean pane and wait until it discovers `retry_b` and `retry_consumer`. Work on B only for now. Leave the consumer unformalized until Section 7.
3. Click **Formalize** for `retry_b`. Wait for the attempt to settle, even if it ends with a review or incomplete outcome. This establishes the target's formalization, file association, and conversation.
4. Open B's Lean code and note its exact file path, module import path, full declaration name, and session link. Use the file displayed by the app; do not guess a directory from the Overleaf project title.
5. Confirm that the selected target is named `retry_b`, possibly under a namespace. If the model chose another name, retain that exact name in the fixture and all later snippets. Do not create a second declaration for B.

2.1 Namespace rule for every snippet

The supplied Lean fixtures use `namespace Lea.RetryLab` as a concrete example. Replace that wrapper with the namespace actually used by B's generated declaration. If B is top-level, omit both the opening `namespace` and closing `end` lines. Keep the target's full name unchanged.

Record: Project name; namespace; B's declaration, path, and module; formalization and session IDs if available; model and allowances. This is the identity baseline.

3 Install the challenging shared file

A completed declaration appears before B; an unrelated unfinished one appears after it.

1. In B's Lean pane, click **Edit** and replace its file contents with `fixtures/shared-baseline.lean`, adapting the namespace/name as above:

```
namespace Lea.RetryLab

-- KEEP: handwritten context, ticket 731.
def kept_def : Nat := 7

theorem kept_a (n : Nat) : n + 0 = n := by
  exact Nat.add_zero n

theorem retry_b (n : Nat) : n + 0 = n := by
  have hn : n = n := by rfl
  sorry

-- KEEP: unrelated work must remain unfinished.
theorem untouched_c (n : Nat) : n + 0 = n := by
  sorry

end Lea.RetryLab
```

2. Click **Save**. Compilation may succeed with `sorry` warnings. B must remain unfinished. If the app already calls B proved merely because `kept_a` passed, record a target-selection failure.

3. Copy the complete saved text into an external baseline file or note. Verify it contains both `KEEP` comments, `kept_def = 7`, A's original proof, B's `have` line, and C's `sorry`.

4. If comfortable locating the displayed file on disk, add this comment while no run is active:

   ```
   -- KEEP: external manual edit before retry, ticket 731.
   ```

   Save with your editor and do not commit it. A historical check may become stale; that is expected. The retry must capture current bytes. Refresh/reopen the code and confirm the line is present before continuing. If the app cannot load the external revision, record that separately.

5. Keep the actual current file as the baseline. Optionally add an untracked `manual-notes.txt` beside it containing `KEEP: handwritten experiment notes`. Record its bytes too.

Why this is challenging: A shared-file guard can reject B before solving. A whole-file `sorry` rule can prevent completion because of C. A first-declaration heuristic can certify A instead. Resetting to Git HEAD can lose external edits. This fixture exercises all four risks.

4 Retry B and inspect the publication

The essential trial. Run it in Overleaf, then repeat from a fresh baseline in standalone.

1. Select B and invoke its explicit **Re-formalize** action. If the current state presents **Resume**, use the overflow **Re-formalize** action for this trial. Ordinary chat is a different entry point.
2. In standalone, open this same backend project, select B's formalization, and use its explicit re-formalization action. If it offers a request field, use:

   ```
   Complete retry_b in its current shared Lean file.
   Keep its statement and full name. Preserve kept_a,
   kept_def, untouched_c, and all KEEP comments.
   Use the existing partial proof where useful.
   Leave unrelated unfinished declarations unfinished.
   ```

   The Overleaf button already supplies the target; it need not expose a custom prompt field.
3. Observe admission. The operation may queue, then start. It must not stop with a shared-file refusal or a request that you split the file yourself. Open the existing target conversation and confirm its earlier history remains.
4. Let B settle. Open the *current published* Lean source and compare it with your saved baseline. Check B's actual declaration, not just a general *compiles* badge.
5. Verify that B has a complete proof, for example:

   ```
   theorem retry_b (n : Nat) : n + 0 = n := by
     have hn : n = n := by rfl
     exact Nat.add_zero n
   ```

   A different valid proof is acceptable. The old partial target proof must remain recoverable even if the solver rewrites it.
6. Confirm C still has `sorry`; A's proof, the definition body, and both `KEEP` comments are unchanged. Any external comment and notes must survive.
7. Reload the browser and select B again. It must reattach to the same target and conversation and show the settled result accurately.

Pass: B is complete beside unfinished C, with fresh selected-target evidence. Protected work is retained. A checked mechanical relocation is acceptable if imports and mappings remain correct and the app handles it automatically.

Fail: B is blocked for sharing a file; C is edited without authorization; A's proof/comments disappear; B is called complete while its trust closure contains `sorry`; or reload loses the operation/history.

Record: Before/after code; outcome; target/conversation identity; elapsed time; model cost; any relocation and its updated path.

5 Complete an existing needed helper

B already depends on the unfinished helper before admission.

1. With no run active, edit B's existing associated file to the supplied helper baseline, `fixtures/helper-baseline.lean`:

```
namespace Lea.RetryLab

-- KEEP: handwritten context, ticket 731.
def kept_def : Nat := 7

theorem kept_a (n : Nat) : n + 0 = n := by
  exact Nat.add_zero n

private theorem hidden_helper (n : Nat) : n + 0 = n := by
  sorry

theorem retry_b (n : Nat) : n + 0 = n := by
  exact hidden_helper n

-- KEEP: unrelated work must remain unfinished.
theorem untouched_c (n : Nat) : n + 0 = n := by
  sorry

end Lea.RetryLab
```

2. Save and confirm B remains unfinished. Although B has no literal `sorry`, it depends on `hidden_helper`, whose proof does.
3. Copy the file as the new before-image. Invoke B's explicit **Re-formalize** action. If a prompt is available, ask it to complete the existing `hidden_helper` under its existing type and finish B, preserving A, C, and the comments.
4. Inspect the source and evidence. The helper can become:

   ```
   private theorem hidden_helper (n : Nat) : n + 0 = n := by
     exact Nat.add_zero n
   ```

   B may continue to use `exact hidden_helper n`. No duplicate helper or weakened target statement is needed.
5. Verify A and C remain unchanged and B's selected trust closure no longer contains `sorryAx`.

Why this is challenging: The original B dependency establishes that this helper belongs to its existing proof work. Adding `exact untouched_c n` to a new candidate does not grant permission to rewrite unrelated C. Private declarations also exercise compiler identity and result-selection handling.

Pass: The eligible helper is completed without changing its type or visibility, and B is certified only after that dependency is sound.

Fail: B receives a proved label while the helper still uses `sorry`; Lea requires a manual split; or unrelated C is treated as an editable helper.

Record: Before/after helper and B; protected-neighbor comparison; final outcome. Section 9 separately tests independently published shared helpers surviving parent failure; this local helper recipe does not establish that stronger case.

6 Retain drafts, stop safely, and detect conflict

Two separate trials. Restore the shared baseline before each.

6.1 Trial A: stop after useful staged progress

1. Restore Section 3's baseline and start B with **Re-formalize**.
2. Wait until the timeline shows a Lean file written in the attempt. Click **Stop**, or **Stop reformalization** when shown. Do not merely close the tab.
3. Wait for Stop to settle. Inspect the published source and retained draft. In standalone, look for **Retained reformalization draft** and **Compare with retained baseline**.
4. Reload. The draft must remain inspectable and the conversation must remain. Reload must not itself start a fresh solver.
5. Explicitly choose continuation or a new re-formalization. It must remain associated with B. A new retry captures current canonical work; a clean Resume uses its compatible checkpoint.

Pass: Unfinished target edits do not replace canonical work; captured drafts survive; no late automatic wakeup undoes Stop. If publication completed before Stop arrived, keeping that settled result is correct. Record *inconclusive for staged cancellation* and use Section 9.

6.2 Trial B: change canonical bytes while B is active

1. Restore the shared baseline, start B, and wait until the operation has prepared its stage or begun a solver tool call.
2. In an external editor, add this line to the canonical file whose path you recorded, then save:

   ```
   -- KEEP: external edit while retry is active, ticket 947.
   ```

   Use an external editor: the app's coordinated Edit action may correctly refuse a write while the project is busy.
3. Let the retry settle. Reopen the current source and inspect retained candidate/conflict information.

Pass: An observed divergent version blocks blind application; the human edit and candidate are retained. An edit before stage capture can legitimately enter the baseline; that does not test an application conflict.

Fail: An observed external edit is silently discarded or an incomplete candidate is presented as current published proof.

Record: Stage-capture and external-save timing; observed versions; result. Use Section 9 for repeatable boundary injection. This manual interleaving does not establish universal exclusion of external editors.

7 Change the source contract and expose a consumer

A source update can succeed while a dependent correctly needs repair.

1. Return to completed B with its original type `(n : Nat) : n + 0 = n`. Keep A and C in the shared file.
2. Formalize `retry_consumer` from the source fixture. Once settled, click **Edit** for the consumer and install:

   ```
   import Lea.RetryLab.retry_b
   namespace Lea.RetryLab
   theorem retry_consumer (n : Nat) : n + 0 = n := by
     exact retry_b n
   end Lea.RetryLab
   ```

   Replace the import with B's actual module path and adapt the namespace. Save and confirm the consumer passes. A consumer with an independent proof does not exercise this case.
3. Save copies of both files. In Overleaf *source*, change only B's theorem sentence to:

   ```
   For every natural number $n$, both $n+0=n$ and $n=n$.
   ```

   Keep its `% lea:` directive and label unchanged. Leave the consumer's source sentence unchanged.
4. Wait for source inventory refresh. B should become out of date. Invoke B's **Re-formalize** action to capture current source.
5. Inspect the result. A corresponding contract and proof are:

   ```
   theorem retry_b (n : Nat) : And (n + 0 = n) (n = n) := by
     exact And.intro (Nat.add_zero n) rfl
   ```

6. Inspect the consumer. Its old `exact retry_b n` now supplies a conjunction where an equality is expected. Its source/proof must remain recoverable, with a current failure or repair need reported honestly.
7. Check A and C again. If extraction was needed, follow the updated target path and imports. Do not reorganize files yourself or request consumer repair until you have recorded this state.

Pass: The checked source-required B update can publish, unrelated work is preserved, and consumer repair need is visible. No consumer success or human approval is invented; no unrequested paid repair begins.

Fail: An ordinary proof retry changes B's interface without changed source; consumer text disappears; or the incompatible dependent remains currently verified.

Record: Old/new source and contract; B's association; consumer bytes and status; A/C comparison. A retained candidate caused by another diagnosed conflict demonstrates protection, but not completed source-update acceptance.

8 Exercise the freshness boundaries

Run from a terminal; these steps do not call a model.

The commands in Sections 8–9 use the repository's provisioned local Python environment. From the monorepo root:

```
cd apps/lea-standalone/adapter
```

8.1 A. Historical validity versus active execution

1. Run the new boundary file and keep verbose output:

```
./.venv/bin/python -m pytest \
  tests/test_freshness_boundaries.py -v
```

2. Check the identical-input cases: chmod, same-byte rewrite, and atomic replacement of configuration, source, compiled artifact, and compiler bytes. Historical evidence should stay current.
3. Check the same-size edit with restored modification time. It must be stale; size/mtime cannot substitute for hashing actual bytes.
4. Check changed-then-restored active checks and kernel replay. They must reject that execution: a compiler could have seen intermediate bytes.
5. Check failed/cancelled admission and partial-build cases. Potentially loaded changed artifacts must retire the serving generation even if the request/build does not succeed.

These tests stub compiler/runtime boundaries to force precise interleavings. They test the policy; the next command exercises actual local Lean.

8.2 B. Real Lean, warm reuse, and changed imports

1. Run:

```
./.venv/bin/python -m pytest \
  tests/test_check_evidence.py -v
```

2. Confirm these real-Lean cases execute and pass:

```
test_new_imports_are_built_and_transitive_changes_are_detected
test_unchanged_warm_check_reuses_environment
test_reordered_imports_new_module_and_noop_rebuild_keep_real_daemon
```

3. For warm/no-op cases, require the same `execution.daemon_generation`. A genuinely changed loaded artifact must cause retirement before that stale generation can serve new work.

Why this is challenging: An overly conservative implementation rejects safe evidence or restarts Lean on every rebuild. An overly permissive one can certify mixed inputs or reuse a server holding stale artifacts. Both directions matter.

Record: Exit code; passed/failed/skipped counts; failing case; generation and `timings_ms` when available. Do not infer reuse from a fast spinner.

9 Force queue, approval, and recovery boundaries

Use deterministic tests when a quick theorem makes manual timing unreliable.

Stay in `apps/lea-standalone/adapter`. Commands use existing disposable fixtures. Keep `-rs` output so unavailable executor cases remain visible.

9.1 A. Two retries in the same file

```
./.venv/bin/python -m pytest \
  tests/test_reformalization_operations.py -v -rs \
  -k "fifo_second_capture or managed_siblings"
```

Pass: The second capture sees the first publication. Managed siblings serialize, and helper receipts refresh later inputs, including recovery after acceptance.

9.2 B. Cancel and retain useful helper work

```
./.venv/bin/python -m pytest \
  tests/test_reformalization_operations.py -v -rs \
  -k "cancel or helper_only or inline_helper"
```

Pass: Cancellation cannot overwrite accepted receipts or revive stopped waits. A separately verified published helper survives parent failure; helper-only publication does not leak unfinished target edits.

9.3 C. Approve the exact draft, then test drift

```
./.venv/bin/python -m pytest \
  tests/test_reformalization_operations.py -v -rs \
  -k "review_applies_exact_draft or review_publication"
```

Pass: Approval queues the exact candidate, applies without another solver run, and rejects a changed draft. Approval joins FIFO at approval time. Withdrawal changes approval state without rewriting classifier history.

9.4 D. Crash and external-file interleavings

```
./.venv/bin/python -m pytest \
  tests/test_reformalization_operations.py -v -rs \
  -k "recovery or restore or descriptor or exact_commit"
```

Pass: Repeated recovery is idempotent; captured and observed external versions remain recoverable; unrelated staged and unstaged Git work survives.

For complete coverage, including definition, disproof, relocation, and protection-rejection cases:

```
./.venv/bin/python -m pytest \
  tests/test_reformalization_operations.py \
  tests/test_reformalization_scope.py -q -rs
```

Executor-dependent skips do not establish that confinement or recovery passed.

10 Compare versions and record the outcome

Keep correctness and usability evidence separate.

10.1 Before/after procedure

1. Compare `8d92209` before the two commits, `0ca2f3c` after them, and the current working tree including uncommitted edits. Save the current diff and untracked files before preparing a historical checkout.
2. Run each state in a separate disposable checkout and app-data/workspace copy. Use identical source/Lean fixtures, toolchain, model, and limits. Never point an older adapter at the live database or reset the active project to switch versions.
3. Run the essential UI recipe for each state. Record an old shared-file refusal as reproduced original behavior. Keep UI outcomes separate from tests introduced only in newer revisions.
4. Repeat the model trial three times from the same initial fixture per state. Do not reuse the previous trial's completed proof. Use deterministic operation tests to isolate application correctness from solver variation.
5. Compare completion rate, collateral edits, false completion/staleness, reload/continuation, and cost. Measure warm latency separately from the first cold check. Correctness gates come before speed claims.

10.2 Result sheet

Field	Entry
Code state / diff identity	
Runtime / executor available	
Project / B identity / session	
Model / limits / trial number	
B admitted and completed?	
A, C, comments, notes kept?	
Helper / consumer outcome	
Draft / conflict / reload	
Generation / elapsed / cost	
Tests passed / failed / skipped	
Evidence folder / screenshots	
Verdict / uncertainty	

10.3 Acceptance rule

Call the shared-file change effective when B repeatedly completes where it previously blocked, protected work is preserved, and incomplete/stale results are never presented as current completion. Call freshness effective when same-byte inputs preserve historical validity and healthy daemon reuse, while changed inputs invalidate evidence and unstable active checks reject.

Preparation evidence: the earlier review ran 131 focused backend tests and 80 focused client tests successfully. Those counts exclude a full operation-suite run and paid model trials. This guide is a procedure, not a claim that its manual recipes have already passed.